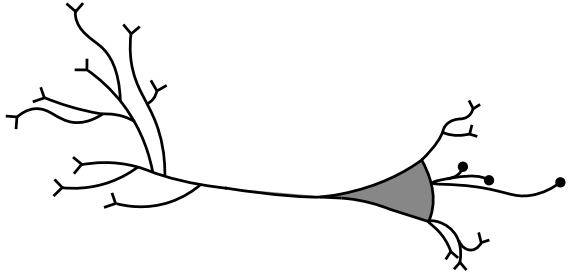
Attractors, memory and canonical computation

Wits Computational Neuroscience COMS4070A / COMS7081A: Lecture 6

August 25, 2026

Kira M. Düsterwald



kira.dusterwald.21@ucl.ac.uk

Where we are and where we are going

Lectures 4–5

- Neural coding and representations
- Recurrent connectivity
- Excitation–inhibition
- Population dynamics

Where we are and where we are going

Lectures 4–5

- Neural coding and representations
- Recurrent connectivity
- Excitation–inhibition
- Population dynamics

Today

- Stable states and attractors
- Memory, decisions and pattern completion
- Hopfield networks
- Canonical computations and compositionality

Where we are and where we are going

Lectures 4–5

- Neural coding and representations
- Recurrent connectivity
- Excitation–inhibition
- Population dynamics

Today

- Stable states and attractors
- Memory, decisions and pattern completion
- Hopfield networks
- Canonical computations and compositionality

recurrency $\longrightarrow$ dynamics $\longrightarrow$ computation

Recap: Recurrence and E-I balance shape dynamics

$$\tau \frac{d\boldsymbol{r}}{dt} = -\boldsymbol{r} + \phi(W\boldsymbol{r} + \boldsymbol{I})$$

Recap: Recurrence and E-I balance shape dynamics

$$\tau \frac{d\boldsymbol{r}}{dt} = -\boldsymbol{r} + \phi(W\boldsymbol{r} + \boldsymbol{I})$$

Recurrent excitation

$E \uparrow \rightarrow E \uparrow$

amplifier + longer timescale

Inhibition

$E \uparrow \rightarrow I \uparrow \rightarrow E \downarrow$

stabilisation + competition

Trajectories

$\boldsymbol{r}(t)$

population state/computation

Recap: Linear vs non-linear dynamical system

With $\phi(x) = x$,

$$\tau \dot{\boldsymbol{r}} = A\boldsymbol{r} + \boldsymbol{I}.$$

This made the system easy to analyse:

nullclines $\rightarrow$ F.P. $\rightarrow$ vector field $\rightarrow$ stability

Recap: Linear vs non-linear dynamical system

With $\phi(x) = x$,

$$\tau \dot{\boldsymbol{r}} = A\boldsymbol{r} + \boldsymbol{I}.$$

This made the system easy to analyse:

nullclines $\rightarrow$ F.P. $\rightarrow$ vector field $\rightarrow$ stability

But neural input–output functions are **non-linear**.

$$\boxed{\tau \dot{\boldsymbol{r}} = -\boldsymbol{r} + \phi(W\boldsymbol{r} + \boldsymbol{I})}$$

Recap: Linear vs non-linear dynamical system

With $\phi(x) = x$,

$$\tau \dot{\boldsymbol{r}} = A\boldsymbol{r} + \boldsymbol{I}.$$

This made the system easy to analyse:

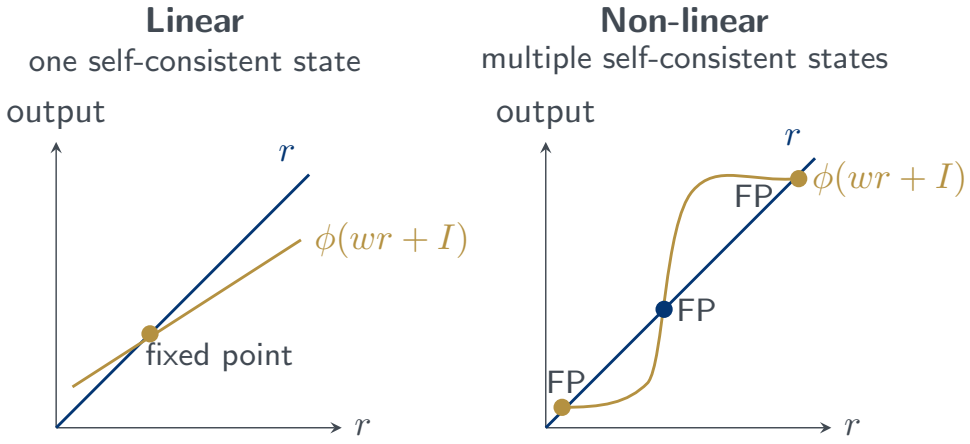
nullclines $\rightarrow$ F.P. $\rightarrow$ vector field $\rightarrow$ stability

But neural input–output functions are **non-linear**.

$$\boxed{\tau \dot{\boldsymbol{r}} = -\boldsymbol{r} + \phi(W\boldsymbol{r} + \boldsymbol{I})}$$

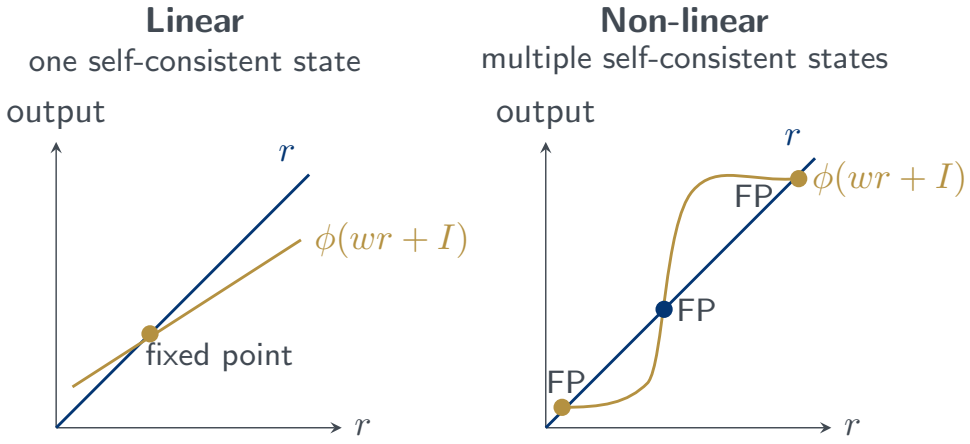
What does this non-linearity actually *buy us*?

What does a non-linearity add?



A non-linear input–output function can produce **multiple** fixed points.

What does a non-linearity add?



A non-linear input–output function can produce **multiple** fixed points.

But how do we classify these fixed points?

Having several fixed points is not the same as having several *stable states*...

Some fixed points are stable; others are not



Stable fixed point

small perturbation $\longrightarrow$ returns

Unstable fixed point

small perturbation $\longrightarrow$ moves away

Some fixed points are stable; others are not



Stable fixed point

small perturbation $\longrightarrow$ returns

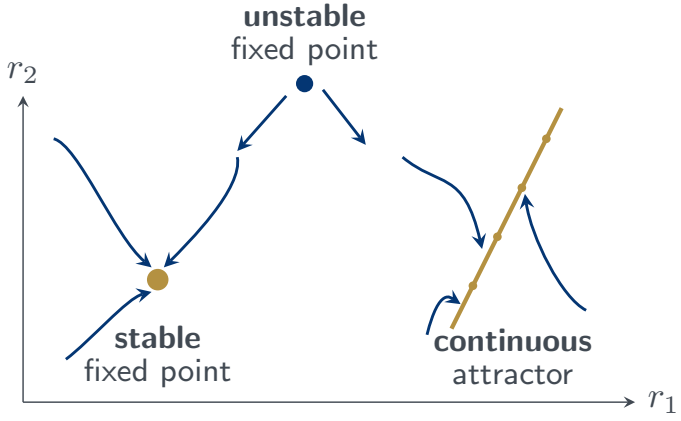
Unstable fixed point

small perturbation $\longrightarrow$ moves away

Different initial states can converge to **different stable states**, called **attractors**.

Attractors and basins of attraction

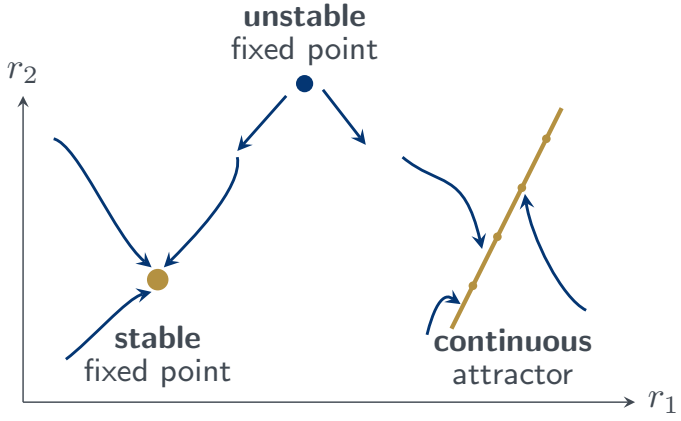
attractor = stable state or set of states toward which nearby trajectories converge

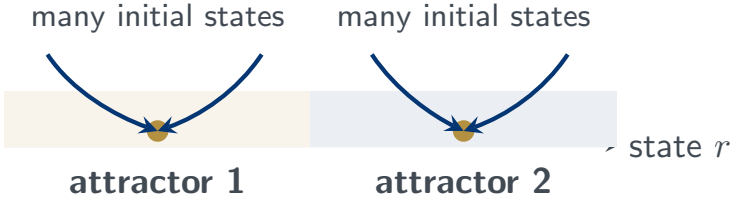


Attractors and basins of attraction

attractor = stable state or set of states toward which nearby trajectories converge

basins of attraction = sets of initial states pulled toward different attractors

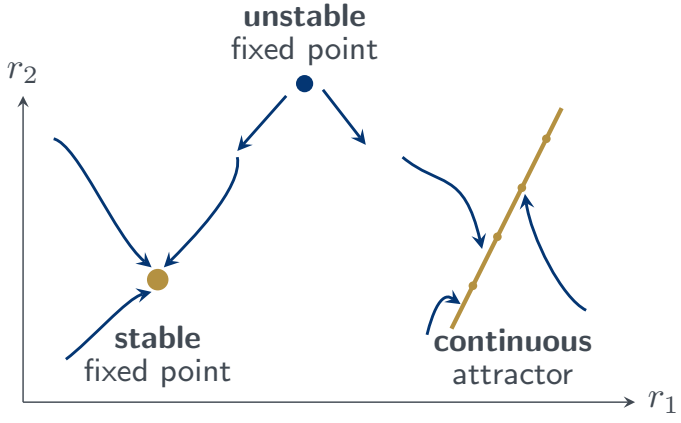


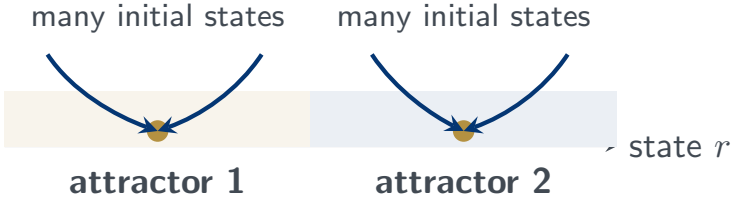


Attractors and basins of attraction

attractor = stable state or set of states toward which nearby trajectories converge

basins of attraction = sets of initial states pulled toward different attractors





Could these stable states represent something?

What could an attractor *mean* computationally?

An attractor is a state that the network **keeps returning to** *even once the cue (external input) disappears*:

$$\boldsymbol{r}(t_0) \longrightarrow \boldsymbol{r}(t) \longrightarrow \boxed{\boldsymbol{r}^*}$$

What could an attractor *mean* computationally?

An attractor is a state that the network **keeps returning to** *even once the cue (external input) disappears*:

$$\boldsymbol{r}(t_0) \longrightarrow \boldsymbol{r}(t) \longrightarrow \boxed{\boldsymbol{r}^*}$$

What if $\boldsymbol{r}^*$ is a **representation of something?**

What could an attractor *mean* computationally?

An attractor is a state that the network **keeps returning to** *even once the cue (external input) disappears*:

$$\boldsymbol{r}(t_0) \longrightarrow \boldsymbol{r}(t) \longrightarrow \boxed{\boldsymbol{r}^*}$$

What if $\boldsymbol{r}^*$ is a **representation of something?**

A *memory* does not have to be a static object stored somewhere. It can be a **stable pattern of activity in a recurrent network**.

What could an attractor *mean* computationally?

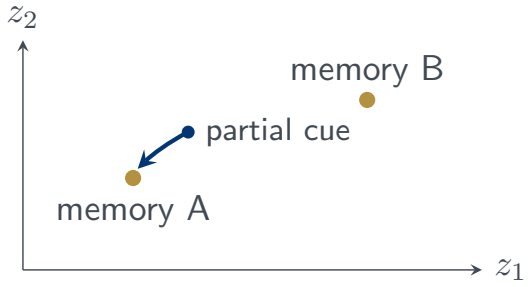
An attractor is a state that the network **keeps returning to** *even once the cue (external input) disappears*:

$$\boldsymbol{r}(t_0) \longrightarrow \boldsymbol{r}(t) \longrightarrow \boxed{\boldsymbol{r}^*}$$

What if $\boldsymbol{r}^*$ is a **representation of something?**

A *memory* does not have to be a static object stored somewhere. It can be a **stable pattern of activity in a recurrent network**.

And what does a *basin* buy us?



What could an attractor *mean* computationally?

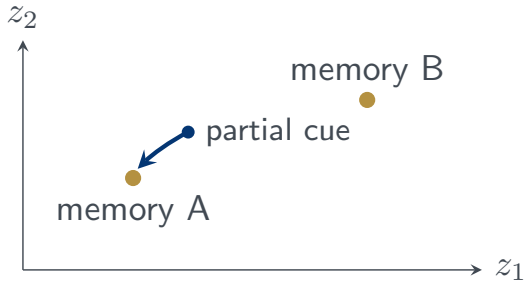
An attractor is a state that the network **keeps returning to** *even once the cue (external input) disappears*:

$$\boldsymbol{r}(t_0) \longrightarrow \boldsymbol{r}(t) \longrightarrow \boxed{\boldsymbol{r}^*}$$

What if $\boldsymbol{r}^*$ is a **representation of something?**

A *memory* does not have to be a static object stored somewhere. It can be a **stable pattern of activity in a recurrent network**.

And what does a *basin* buy us?



The cue does *not* need to specify the whole state! I.e. We get robustness — pattern completion — for free. This is called **associative memory**.

Not all representations are point attractors

This idea seems most natural when thought of as **discrete memories**:

memory A memory B memory C

Not all representations are point attractors

This idea seems most natural when thought of as **discrete memories**:

memory A memory B memory C

But many variables that the brain needs to maintain are **continuous**:

eye position	head direction	spatial position

Not all representations are point attractors

This idea seems most natural when thought of as **discrete memories**:

memory A memory B memory C

But many variables that the brain needs to maintain are **continuous**:

eye position	head direction	spatial position

Instead of isolated point attractors:

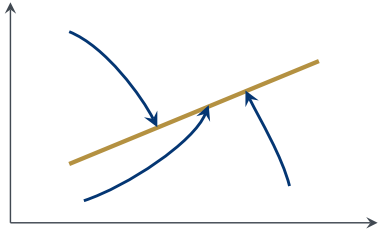
• • •

we can have a **continuous manifold of stable states**:

line attractor ring attractor plane attractor

Line and ring attractors: Examples in the brain

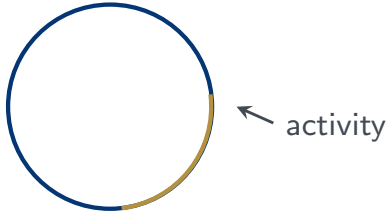
Line attractor



Eye-position memory:
The neural integrator can maintain different eye positions after a saccade.

Seung, 1996

Ring attractor

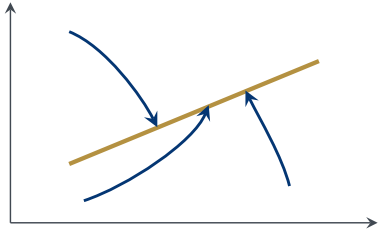


Head-direction cells:
The bump marks the animal's current heading and can move smoothly around the ring.

Knierim and Zhang, 2012

Line and ring attractors: Examples in the brain

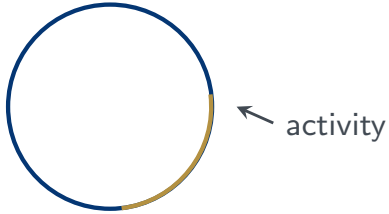
Line attractor



Eye-position memory:
The neural integrator can maintain different eye positions after a saccade.

Seung, 1996

Ring attractor



Head-direction cells:
The bump marks the animal's current heading and can move smoothly around the ring.

Knierim and Zhang, 2012

The geometry of the attractor reflects the variable being represented.

Attractors do not only represent memories: Decision-making

The same dynamical machinery can implement a choice.

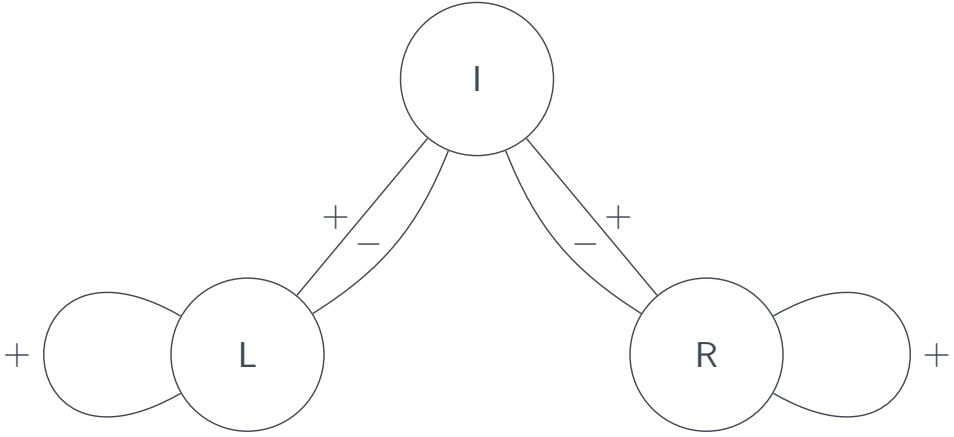
Attractors do not only represent memories: Decision-making

The same dynamical machinery can implement a choice.

Memory		**Decision**
$A =$ stored pattern 1	$\longrightarrow$	$A =$ choice L
$B =$ stored pattern 2		$B =$ choice R

Memory retrieves a representation, while a decision must **select one action** under uncertainty.

The **winner-take-all** network for decisions



$$E_L \uparrow\!\rightarrow I \uparrow\!\rightarrow E_R \downarrow$$

One population wins; the other is suppressed.

Kaski and Kohonen, *Neural Networks*, 1994

How can a recurrent circuit create useful computation?

A (memory) attractor requires **three ingredients**:

Positive feedback / gain

$$E\uparrow \longrightarrow E\uparrow$$

activity reinforces itself

Stabilisation

$$E\uparrow \longrightarrow I\uparrow \longrightarrow E\downarrow$$

prevents runaway activity

Competition

$$E_A\uparrow \longrightarrow E_B\downarrow$$

different states compete

How can a recurrent circuit create useful computation?

A (memory) attractor requires **three ingredients**:

Positive feedback / gain

$$E\uparrow \longrightarrow E\uparrow$$

activity reinforces itself

Stabilisation

$$E\uparrow \longrightarrow I\uparrow \longrightarrow E\downarrow$$

prevents runaway activity

Competition

$$E_A\uparrow \longrightarrow E_B\downarrow$$

different states compete

recurrent excitation + E–I stabilisation + competition $\longrightarrow$ stable, selective population states

How can a recurrent circuit create useful computation?

WITS UNIVERSITY

A (memory) attractor requires **three ingredients**:

Positive feedback / gain

$$E \uparrow \longrightarrow E \uparrow$$

activity reinforces itself

Stabilisation

$$E \uparrow \longrightarrow I \uparrow \longrightarrow E \downarrow$$

prevents runaway activity

Competition

$$E_A \uparrow \longrightarrow E_B \downarrow$$

different states compete

recurrent excitation + E–I stabilisation + competition $\longrightarrow$ stable, selective population states

This is what makes recurrence computationally useful.

Question: Can we design the recurrent connectivity so that the **stable states are memories?**

Hopfield's idea: build the attractors into the weights

Can we choose W so that desired patterns are stable states?

- Store a set of binary patterns across all neurons $\xi^1, \xi^2, \ldots, \xi^P$
- Construct recurrent weights from those patterns
- Implement Hebbian learning
- Let the network dynamics retrieve a stored pattern from a noisy cue

This is a minimal model of **distributed associative memory**.

Hopfield, *PNAS*, 1982; Amit and Brunel, *Cereb. Cortex*, 1997

The classical Hopfield network

Let each neuron take a binary state:

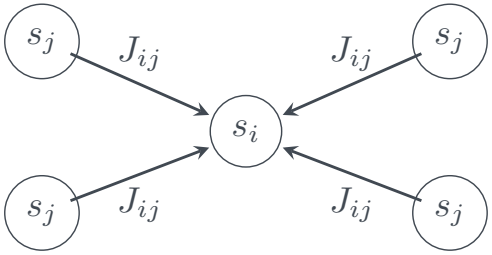
$$s_i \in \{-1, +1\}.$$

A simple update rule is

$$s_i(t+1) = \text{sign}\left(\sum_j J_{ij} s_j(t)\right)$$

where the total (dense) recurrent input to a neuron i is

$$h_i = \sum_j J_{ij} s_j.$$



The classical Hopfield network

Let each neuron take a binary state:

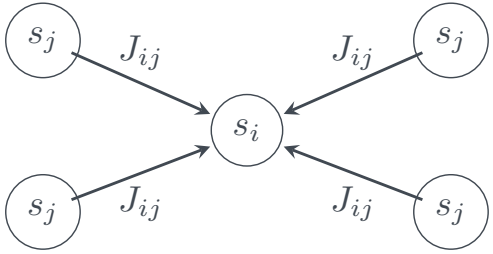
$$s_i \in \{-1, +1\}.$$

A simple update rule is

$$\boxed{s_i(t+1) = \text{sign}\left(\sum_j J_{ij} s_j(t)\right)}$$

where the total (dense) recurrent input to a neuron i is

$$h_i = \sum_j J_{ij} s_j.$$



1. How do we store memories?
2. Why are memories stable?
3. Is there a storage capacity bound?
4. Energy landscape intuition
5. Biological plausibility

1. How do we choose weights to store desired memories?

WITS UNIVERSITY

Suppose the desired memories are

$$\xi^1, \xi^2, \ldots, \xi^P, \qquad \xi_i^\mu \in \{-1, +1\}.$$



A Hebbian update construction is

$$J_{ij} \leftarrow \frac{1}{N} \sum_{\mu=1}^{P} \xi_i^\mu \xi_j^\mu, \qquad J_{ii} = 0.$$

$\Delta w_{ij} = \eta x_i y_j$

"fire together, wire together"

1. How do we choose weights to store desired memories?

Suppose the desired memories are

$$\xi^1, \xi^2, \ldots, \xi^P, \qquad \xi_i^\mu \in \{-1, +1\}.$$

A Hebbian update construction is

$$J_{ij} \leftarrow \frac{1}{N} \sum_{\mu=1}^{P} \xi_i^\mu \xi_j^\mu, \qquad J_{ii} = 0.$$

Interpretation

Only neurons that are active together in a stored pattern receive a *positive* mutual connection ("neurons that fire together wire together").

Quick exercise: what happens on the next update?

Suppose we store two patterns:

$$\xi^1 = (+1, +1, +1, -1, -1, -1) \qquad \xi^2 = (+1, -1, -1, +1, -1, +1)$$

and the network is currently in state

$$s = (+1, +1, -1, -1, -1, +1).$$

Question

For neuron $i = 3$, what is the sign of its next state (using the Hebbian rule)?

$$s_3(t+1) = \text{sign}\left(\sum_{j \neq i} J_{3j} s_j\right)$$

$$J_{ij} = \frac{1}{N} \sum_{\mu} \xi_i^\mu \xi_j^\mu$$

$$J_{31} = \frac{1}{2}\big((1)(1) + (1)(-1)\big) = 0$$

Quick exercise: what happens on the next update? (Answer)

WITS UNIVERSITY

For neuron 3,

$$h_3 = \sum_j J_{3j} s_j.$$

The Hebbian weights from neuron 3 are

$$J_{31} = \frac{1}{6}(1 \cdot 1 + (-1) \cdot 1) = 0,$$
$$J_{32} = \frac{1}{6}(1 \cdot 1 + (-1) \cdot (-1)) = \frac{1}{3},$$
$$J_{34} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot 1) = -\frac{1}{3},$$
$$J_{35} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot (-1)) = 0,$$
$$J_{36} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot 1) = -\frac{1}{3}.$$

Quick exercise: what happens on the next update? (Answer)

For neuron 3,

$$h_3 = \sum_j J_{3j} s_j.$$

The Hebbian weights from neuron 3 are

$$J_{31} = \frac{1}{6}(1 \cdot 1 + (-1) \cdot 1) = 0,$$
$$J_{32} = \frac{1}{6}(1 \cdot 1 + (-1) \cdot (-1)) = \frac{1}{3},$$
$$J_{34} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot 1) = -\frac{1}{3},$$
$$J_{35} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot (-1)) = 0,$$
$$J_{36} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot 1) = -\frac{1}{3}.$$

Therefore

$$h_3 = 0(+1) + \frac{1}{3}(+1) - \frac{1}{3}(-1) + 0(-1) - \frac{1}{3}(+1)$$
$$= \frac{1}{3}.$$

$$\boxed{s_3(t+1) = \text{sign}\,(h_3) = +1}$$

What is your interpretation of this?

Clue: Which pattern do we now recover?

Quick exercise: what happens on the next update? (Answer)

For neuron 3,

$$h_3 = \sum_j J_{3j} s_j.$$

The Hebbian weights from neuron 3 are

$$J_{31} = \frac{1}{6}(1 \cdot 1 + (-1) \cdot 1) = 0,$$
$$J_{32} = \frac{1}{6}(1 \cdot 1 + (-1) \cdot (-1)) = \frac{1}{3},$$
$$J_{34} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot 1) = -\frac{1}{3},$$
$$J_{35} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot (-1)) = 0,$$
$$J_{36} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot 1) = -\frac{1}{3}.$$

Therefore

$$h_3 = 0(+1) + \frac{1}{3}(+1) - \frac{1}{3}(-1) + 0(-1) - \frac{1}{3}(+1)$$
$$= \frac{1}{3}.$$

$$\boxed{s_3(t+1) = \text{sign}\,(h_3) = +1}$$

What is your interpretation of this?

Clue: Which pattern do we now recover?

Quick exercise: what happens on the next update? (Answer)

For neuron 3,

$$h_3 = \sum_j J_{3j} s_j.$$

The Hebbian weights from neuron 3 are

$$J_{31} = \frac{1}{6}(1 \cdot 1 + (-1) \cdot 1) = 0,$$
$$J_{32} = \frac{1}{6}(1 \cdot 1 + (-1) \cdot (-1)) = \frac{1}{3},$$
$$J_{34} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot 1) = -\frac{1}{3},$$
$$J_{35} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot (-1)) = 0,$$
$$J_{36} = \frac{1}{6}(1 \cdot (-1) + (-1) \cdot 1) = -\frac{1}{3}.$$

Therefore

$$h_3 = 0(+1) + \frac{1}{3}(+1) - \frac{1}{3}(-1) + 0(-1) - \frac{1}{3}(+1)$$
$$= \frac{1}{3}.$$

$$\boxed{s_3(t+1) = \text{sign}\,(h_3) = +1}$$

What is your interpretation of this?

Clue: Which pattern do we now recover?

The network corrects the corrupted bit:

The basin of attraction nudges partial cues to the closest full memory (ξ^2)...

2. Why do stored patterns reinforce under Hebbian updates?

Suppose the network is exactly in memory state ξ^μ.

The input to neuron i is

$$h_i = \sum_j J_{ij} \xi_j^\mu.$$

Substitute the Hebbian weight update:

$$h_i = \frac{1}{N} \sum_j \sum_{\nu=1}^{P} \xi_i^\nu \xi_j^\nu \xi_j^\mu.$$

2. Why do stored patterns reinforce under Hebbian updates?

Suppose the network is exactly in memory state ξ^μ.

The input to neuron i is

$$h_i = \sum_j J_{ij} \xi_j^\mu .$$

Substitute the Hebbian weight update:

$$h_i = \frac{1}{N} \sum_j \sum_{\nu=1}^{P} \xi_i^\nu \xi_j^\nu \xi_j^\mu .$$

Separate the term $\nu = \mu$:

$$h_i = \underbrace{\xi_i^\mu}_{\text{signal}} + \underbrace{\sum_{\nu \neq \mu} \xi_i^\nu \left(\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \right)}_{\text{crosstalk}} .$$

$$\frac{1}{N} \sum_j^N \xi_i^\nu \xi_j^\nu \xi_j^\mu = \frac{1}{N} N \cdot \xi_i^\mu = \xi_i^\mu$$

(1) $\forall j$

2. Why do stored patterns reinforce under Hebbian updates?

Suppose the network is exactly in memory state ξ^μ.

The input to neuron i is

$$h_i = \sum_j J_{ij} \xi_j^\mu.$$

Substitute the Hebbian weight update:

$$h_i = \frac{1}{N} \sum_j \sum_{\nu=1}^{P} \xi_i^\nu \xi_j^\nu \xi_j^\mu.$$

Separate the term $\nu = \mu$:

$$h_i = \underbrace{\xi_i^\mu}_{\text{signal}} + \underbrace{\sum_{\nu \neq \mu} \xi_i^\nu \left(\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \right)}_{\text{crosstalk}}.$$

For $\nu \neq \mu$, the patterns are uncorrelated (**in large pattern limit!*), so

$$\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \approx 0.$$

2. Why do stored patterns reinforce under Hebbian updates?

WITS UNIVERSITY

Suppose the network is exactly in memory state ξ^μ.

The input to neuron i is

$$h_i = \sum_j J_{ij} \xi_j^\mu.$$

Substitute the Hebbian weight update:

$$h_i = \frac{1}{N} \sum_j \sum_{\nu=1}^{P} \xi_i^\nu \xi_j^\nu \xi_j^\mu.$$

Separate the term $\nu = \mu$:

$$h_i = \underbrace{\xi_i^\mu}_{\text{signal}} + \underbrace{\sum_{\nu \neq \mu} \xi_i^\nu \left(\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \right)}_{\text{crosstalk}}.$$

For $\nu \neq \mu$, the patterns are uncorrelated (*in large pattern limit!*), so

$$\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \approx 0.$$

Therefore

$$\boxed{h_i \approx \xi_i^\mu}$$

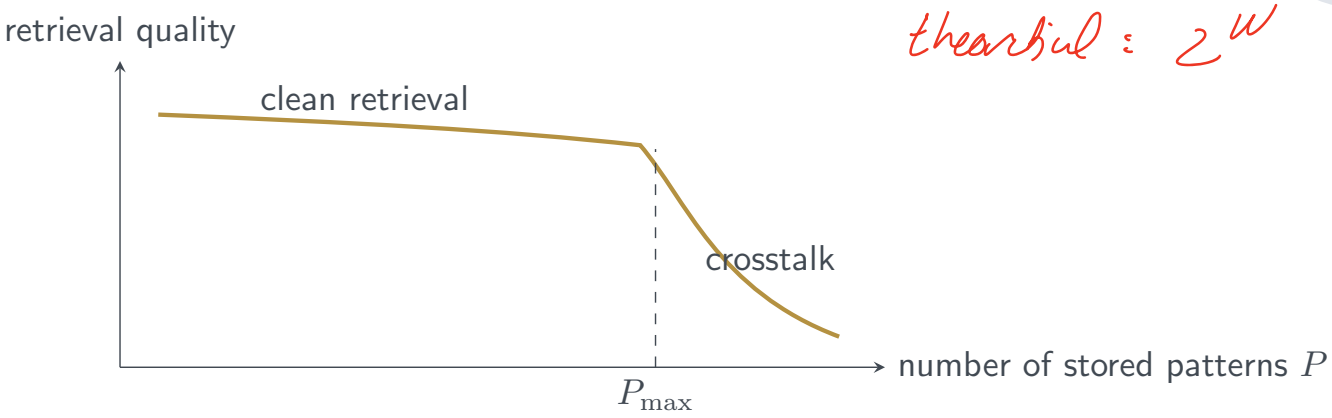
$S_i(t+1) = \text{sign}(h_i) = S_i(t)$

The target pattern adds coherently, while the other patterns average out.

3. What is the bound on storage capacity? (Intuition)

how many memories? P.

3. What is the bound on storage capacity? (Intuition)



theoretical: 2^N

As P increases:

more memories $\rightarrow$ more crosstalk $\rightarrow$ less reliable retrieval

3. What is the bound on storage capacity? (Number)

For random, dense, classical Hopfield patterns, reliable storage is bounded at approx.

$$\boxed{\frac{P_{\max}}{N} \approx 0.138}$$

in the standard large-N regime.

Only an $O(N)$ number of random memories can be stored reliably.

The important idea is the signal-to-crosstalk trade-off

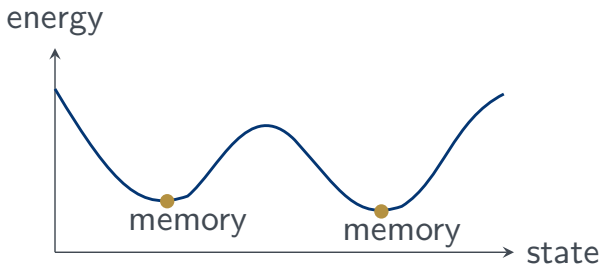
We will not derive this today: The precise capacity comes from statistical-mechanics analysis.

4. The energy landscape formulation

For symmetric weights, $J_{ij} = J_{ji}$, define

$$E(\boldsymbol{s}) = -\frac{1}{2}\sum_{i,j} J_{ij} s_i s_j.$$

Under suitable asynchronous updates, the network moves toward lower energy.



stored pattern $\longleftrightarrow$ stable attractor $\longleftrightarrow$ energy minimum

5. Is the classical Hopfield network biologically plausible?

5. Is the classical Hopfield network biologically plausible?

What it captures

- Distributed representations
- Recurrence and memory
- Pattern completion
- Robustness to noise
- Competition between stored states

5. Is the classical Hopfield network biologically plausible?

What it captures

- Distributed representations
- Recurrence and memory
- Pattern completion
- Robustness to noise
- Competition between stored states

What is unrealistic

- **Limited capacity** : (A) whitening
- **All-to-all connectivity** : (B) sparsity
- **Binary neurons** : (C) continuous
- Symmetric weights
- Discrete updates
- No natural “no memory” state

5. Is the classical Hopfield network biologically plausible?

What it captures

- Distributed representations
- Recurrence and memory
- Pattern completion
- Robustness to noise
- Competition between stored states

What is unrealistic

- **Limited capacity** : (A) whitening
- **All-to-all connectivity** : (B) sparsity
- **Binary neurons** : (C) continuous
- Symmetric weights
- Discrete updates
- No natural "no memory" state

A toy model (but a useful one!)

Hopfield is best understood as a **minimal model of a computational principle**, not a literal cortical model.

(A) Capacity: Why do uncorrelated terms average out?

Suppose we sum products of two random variables:

$$X_1 Y_1 + X_2 Y_2 + \cdots + X_N Y_N.$$

If the variables are centred and independent,

$$\mathbb{E}[X_j] = \mathbb{E}[Y_j] = 0,$$

then

$$\mathbb{E}[X_j Y_j] = \mathbb{E}[X_j]\mathbb{E}[Y_j] = 0.$$

(A) Capacity: Why do uncorrelated terms average out?

Suppose we sum products of two random variables:

$$X_1 Y_1 + X_2 Y_2 + \cdots + X_N Y_N.$$

If the variables are centred and independent,

$$\mathbb{E}[X_j] = \mathbb{E}[Y_j] = 0,$$

then

$$\mathbb{E}[X_j Y_j] = \mathbb{E}[X_j]\mathbb{E}[Y_j] = 0.$$

So, for large N,

$$\boxed{\frac{1}{N}\sum_{j=1}^{N} X_j Y_j \longrightarrow 0}$$

The positive and negative terms cancel on average.

Why is capacity low? Crosstalk degrades reps

Recall the source of interference:

$$h_i = \xi_i^\mu + \sum_{\nu \neq \mu} \xi_i^\nu \underbrace{\left(\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \right)}_{\text{overlap / crosstalk}}.$$

Why is capacity low? Crosstalk degrades reps

Recall the source of interference:

$$h_i = \xi_i^\mu + \sum_{\nu \neq \mu} \xi_i^\nu \underbrace{\left(\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \right)}_{\text{overlap / crosstalk}}.$$

For random memories, different patterns fluctuate and are only **approximately orthogonal**:

$$\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \approx 0, \qquad \nu \neq \mu.$$

Why is capacity low? Crosstalk degrades reps

Recall the source of interference:

$$h_i = \xi_i^\mu + \sum_{\nu \neq \mu} \xi_i^\nu \underbrace{\left(\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \right)}_{\text{overlap / crosstalk}}.$$

For random memories, different patterns fluctuate and are only **approximately orthogonal**:

$$\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \approx 0, \qquad \nu \neq \mu.$$

↑ would like =

finite sampling $\longrightarrow$ residual (non-zero) overlap $\longrightarrow$ crosstalk $\longrightarrow$ capacity limit

Why is capacity low? Crosstalk degrades reps

Recall the source of interference:

$$h_i = \xi_i^\mu + \sum_{\nu \neq \mu} \xi_i^\nu \underbrace{\left(\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \right)}_{\text{overlap / crosstalk}}.$$

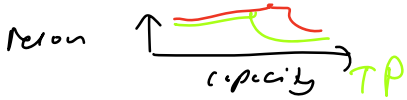
For random memories, different patterns fluctuate and are only **approximately orthogonal**:

$$\frac{1}{N} \sum_j \xi_j^\nu \xi_j^\mu \approx 0, \qquad \nu \neq \mu.$$

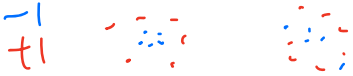
finite sampling $\longrightarrow$ residual (non-zero) overlap $\longrightarrow$ crosstalk $\longrightarrow$ capacity limit

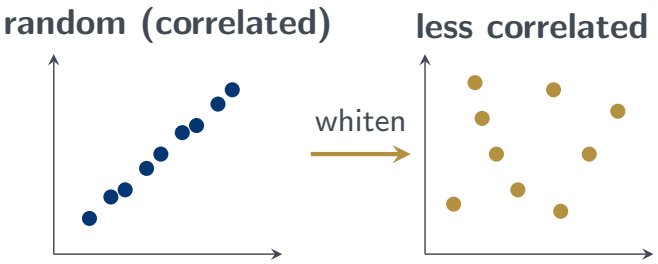
What if we design the representation itself to reduce these overlaps?

Whitening: make the representation less redundant



A useful strategy in neural coding is to **reduce correlations between represented features**.







Whitening: make the representation less redundant

A useful strategy in neural coding is to **reduce correlations between represented features**.



In the idealised limit, we could transform the patterns so that they are **orthogonal**:

$$\boxed{\boldsymbol{\xi}^{\mu} \cdot \boldsymbol{\xi}^{\nu} = 0 \qquad (\mu \neq \nu)}$$

Then the crosstalk term disappears.

Whitening: make the representation less redundant

A useful strategy in neural coding is to **reduce correlations between represented features**.



In the idealised limit, we could transform the patterns so that they are **orthogonal**:

$$\boxed{\boldsymbol{\xi}^{\mu} \cdot \boldsymbol{\xi}^{\nu} = 0 \qquad (\mu \neq \nu)}$$

Then the crosstalk term disappears.

$$\downarrow \text{overlap} \longrightarrow \downarrow \text{crosstalk} \longrightarrow \uparrow \text{capacity}$$

Whitening: make the representation less redundant

A useful strategy in neural coding is to **reduce correlations between represented features**.



In the idealised limit, we could transform the patterns so that they are **orthogonal**:

$$\boxed{\boldsymbol{\xi}^{\mu} \cdot \boldsymbol{\xi}^{\nu} = 0 \qquad (\mu \neq \nu)}$$

Then the crosstalk term disappears.

$$\downarrow \text{overlap} \longrightarrow \downarrow \text{crosstalk} \longrightarrow \uparrow \text{capacity}$$

Whitening / decorrelation can reduce representation redundancy and increase capacity.

Personnaz, Guyon and Dreyfus, 1986; Amit, Gutfreund and Sompolinsky, 1987; Atick and Redlich, 1992

(B) What about connectivity and coding?

hopfield
= dense

Real cortical networks are sparse:

$$K \ll N.$$

If we sparsify the classical Hopfield network, retrieval becomes noisier because each neuron receives fewer supporting inputs.

However, sparse firing patterns can partly compensate for sparse connectivity and can substantially change storage properties...

E.g. Sparse coding

For a given stimulus, only a **small fraction** of neurons are strongly active.

Dense representation

$$\begin{matrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{matrix}$$

many neurons participate

$\rightarrow$

Sparse representation

$$\begin{matrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{matrix}$$

only a few neurons participate

E.g. Sparse coding

For a given stimulus, only a **small fraction** of neurons are strongly active.

Dense representation

$$\begin{matrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{matrix}$$

many neurons participate

$\rightarrow$

Sparse representation

$$\begin{matrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{matrix}$$

only a few neurons participate

Why might this be useful?

A sparse code can represent many different patterns while using relatively little neural activity. "Sparse coding" has been shown to be helpful (+ **efficient!**) for building single-cell receptive fields and representations.

Olshausen and Field, *Nature*, 1996

More broadly, what makes a good representation?

So far we have asked:

What does the neural population represent?

More broadly, what makes a good representation?

So far we have asked:

What does the neural population represent?

But neural systems have limited resources.

What representations use those resources efficiently?

More broadly, what makes a good representation?

So far we have asked:

What does the neural population represent?

But neural systems have limited resources.

What representations use those resources efficiently?

Too much activity
redundant / metabolically costly

$\leftrightarrow$

Too little activity
information may be lost

More broadly, what makes a good representation?

So far we have asked:

What does the neural population represent?

But neural systems have limited resources.

What representations use those resources efficiently?

Too much activity
redundant / metabolically costly

$\leftrightarrow$

Too little activity
information may be lost

Efficient representations balance information and cost.

(C) Continuous states and modern Hopfield networks

Classical Hopfield

$s_i \in \{-1, +1\}$

fixed-point retrieval

Hebbian weights



Modern continuous Hopfield

continuous states

content-addressable retrieval

attention-like updates

query $\longrightarrow$ retrieve relevant stored patterns

Attractor/associative-retrieval ideas appear in modern networks.

Ramsauer et al., *arXiv*, 2021

Zooming out: Where are attractors in the brain?

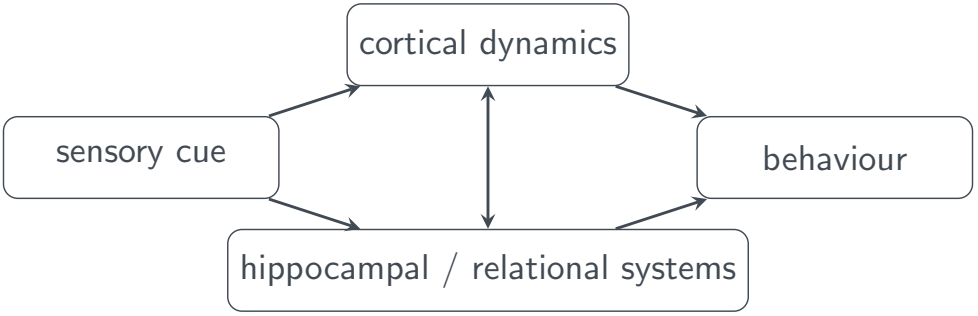
Are attractors properties of

single cells? | local circuits? | whole brain networks?

- The model says an attractor is a property of the **dynamical system**.
- Biological circuits are nested and coupled.
- Different attractors may describe different spatial and temporal scales.

A computational model need not map one-to-one onto an anatomical structure!

Even biological memory is more complicated...



Persistent activity, synaptic plasticity, short-term synaptic dynamics, neuromodulation and interactions between brain systems can all contribute to memory.

Recurrent dynamics' ability to stabilise a representation is just one mechanism.

Memory is not one thing

Working memory

- maintain information over seconds
- can guide current behaviour
- often involves persistent / structured activity

Associative memory

- retrieve a stored representation from a partial cue
- naturally invites a pattern-completion interpretation

Long-term memory

- distributed across multiple systems
- depends on synaptic plasticity as well as network dynamics

We will return to biological memory in more detail later (see Devon's lectures!).

One potential biological grain

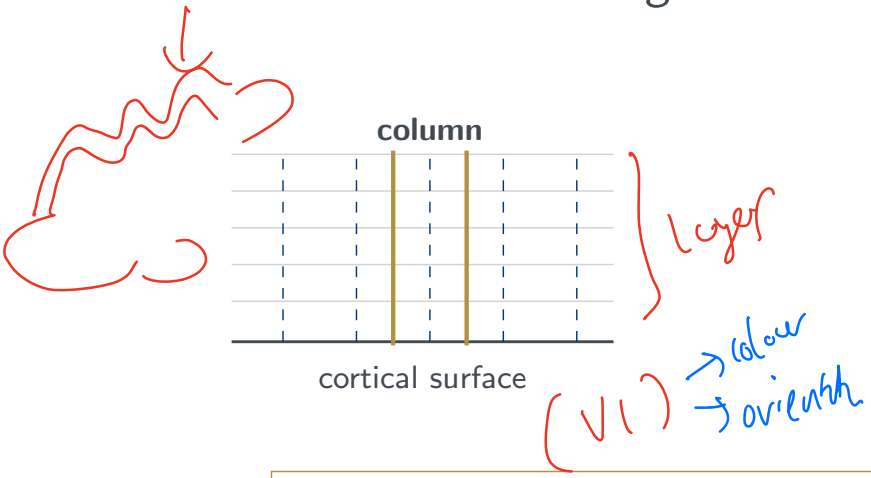
How does this relate to the brain? In 1957, Vernon Mountcastle asked:

How is the cortex organised to perform computation?

One potential biological grain

How does this relate to the brain? In 1957, Vernon Mountcastle asked:

How is the cortex organised to perform computation?



Neurons recorded in a "column" through sensory cortex tend to respond to:

- the same sensory modality
- similar peripheral receptive fields

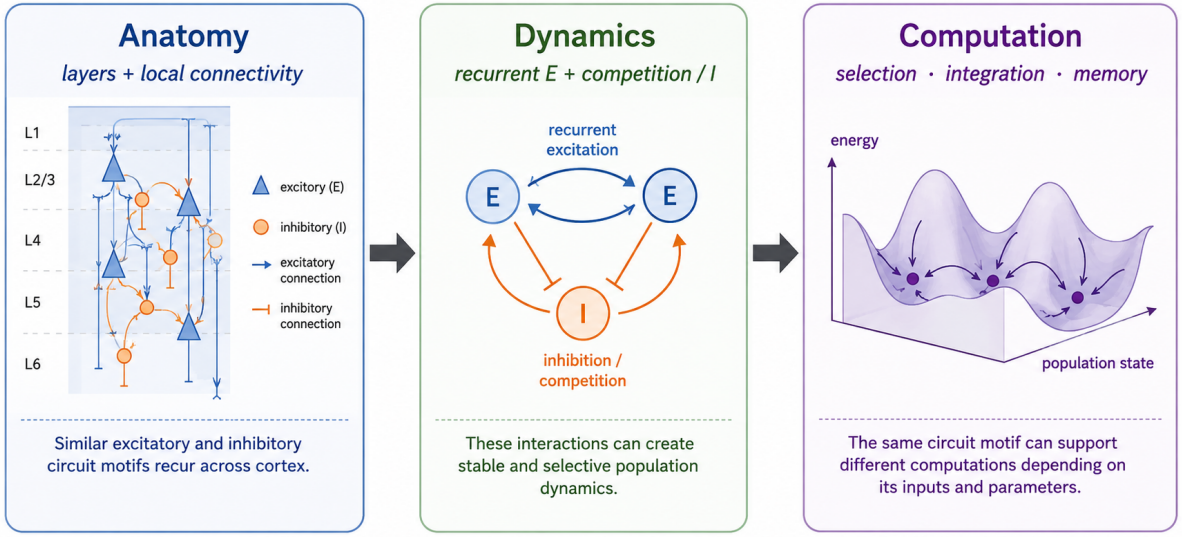
Mountcastle proposed that this **column of neurons** forms an elementary unit of cortical organisation.

Could the cortex be built from repeated local computational units?

Mountcastle, 1957

From cortical columns to canonical microcircuits

Perhaps cortex repeatedly reuses similar circuit motifs (compositionality).



cf. Douglas and Martin, 2004; Image generated by ChatGPT prompt, August 2026

Take home

Question	Idea	Consequence
Why recurrent non-linearities?	multiple stable states	attractors
How can an attractor compute?	stable population pattern	memory / selection
How can memory be stored?	structured recurrent weights	Hopfield network
Why is capacity limited?	crosstalk	try whitening
Bigger picture?	canonical computations	compositionality

If you want to go further:

- Hopfield Network Tutorial (Tom George, TReND CaMinA summer school)
- Original Hopfield paper, 1982
- Modern Hopfield nets: Ramsauer et al., 2021
- Sparse coding: Olshausen and Field, 1996

Lectures 4-6 in context

Question	Neural object	Key idea	Computation
What is represented?	$\boldsymbol{r}$	population code	**representation**
How does it change?	$\boldsymbol{r}(t)$	recurrence + E–I	**dynamics**
Where does it go?	$\boldsymbol{r}^*$	stability + attractors	**memory / selection**

Lectures 4-6 in context

Question	Neural object	Key idea	Computation
What is represented?	$\boldsymbol{r}$	population code	**representation**
How does it change?	$\boldsymbol{r}(t)$	recurrence + E–I	**dynamics**
Where does it go?	$\boldsymbol{r}^*$	stability + attractors	**memory / selection**

representation $\longrightarrow$ dynamics $\longrightarrow$ computation

Lectures 4-6 in context

Question	Neural object	Key idea	Computation
What is represented?	$\boldsymbol{r}$	population code	**representation**
How does it change?	$\boldsymbol{r}(t)$	recurrence + E–I	**dynamics**
Where does it go?	$\boldsymbol{r}^*$	stability + attractors	**memory / selection**

representation $\longrightarrow$ dynamics $\longrightarrow$ computation

The same population can be studied as a **code**, a **dynamical system**, or a **computational device**.